

Unit 8 Study Guide

Polymorphism, Inheritance & Interfaces

Part 1: Constructors & Object Type Assignments

The Two Assignment Rules

Rule 1: The reference type (left side) must be a supertype of the actual type (right side). A subclass object can be stored in a superclass or interface variable. The reverse is a compiler error.

```
Instrument i = new Guitar(); // OK – Guitar is-a Instrument
Guitar g = new Instrument(); // COMPILER ERROR
StringInstrument s = new Guitar(); // OK – Guitar is-a StringInstrument
```

Rule 2: You cannot instantiate an abstract class or an interface.

```
Instrument i = new Instrument(); // COMPILER ERROR – abstract
```

Tracing Constructor Output

When new SubClass(...) is called, Java runs constructors **from the top of the hierarchy down**. Each constructor calls super(...) (explicit or implicit) before its own code runs.

Steps: (1) Identify the constructor called. (2) Follow any this(...) or super(...) call first. (3) Print statements execute after super() returns, in call order.

Example: Guitar g3 = new Guitar();

```
Guitar() → calls this(false)
Guitar(boolean) → calls super(6) [StringInstrument(6)]
    StringInstrument(6) → calls super("String-6") [Instrument("String-6")]
        Instrument("String-6") prints I2-
    StringInstrument(6) prints S1-
Guitar(boolean) prints G2-
Guitar() prints G1-
Output: I2-S1-G2-G1-
```

Instance Variable Values

After construction, check: does this field exist in the *actual object's class or any superclass*? If yes → give its final assigned value. If no → **DNE**.

```
Guitar g3 = new Guitar() → nStrings = 6, electric = false
Instrument i1 = new WindInstrument("Flute", "Silver") → nStrings = DNE, electric = DNE
```

Part 2: instanceof

The Rule

x instanceof T returns **true** if the **actual (runtime) object** is of type T or any subtype of T. It uses the real object, not the declared reference type.

The object is instanceof **everything in its ancestry chain**: its own class, every superclass up to Object, and every interface it (or any ancestor) implements.

How to Solve

(1) Identify the **actual type** (the new ____ part). (2) Trace its chain: actual class → parent → grandparent → Object, plus all interfaces. (3) instanceof T is true if T appears anywhere in that chain.

Example: Loud ld = new Guitar(true); Chain: Guitar → StringInstrument → Instrument → Object, plus Guitar implements Loud.

Check	Result	Reason
ld instanceof Object	true	everything is an Object
ld instanceof Instrument	true	ancestor
ld instanceof StringInstrument	true	parent
ld instanceof Guitar	true	actual type
ld instanceof WindInstrument	FALSE	different branch
ld instanceof Loud	true	Guitar implements Loud

Key trap: The declared type (Loud ld) is irrelevant — only the actual object matters.

Part 3: Casting & Method Calls

Dynamic Dispatch — The Core Rule

The compiler checks the *declared type*. The JVM runs the *actual type*.

- Method not in declared type → **compiler error** (even if actual object has it).
- Method in declared type → compiles; at runtime Java runs the **most overridden version** in the actual class.

```
Instrument ins = new Guitar();
ins.play() // compiles (in Instrument); runs Guitar.play() → "Chord!"
ins.tune() // COMPILER ERROR – tune() is not in Instrument
```

Casting Rules

Downcast ((Guitar) ins) tells the compiler "trust me, this is really a Guitar."

Situation	Result
Declared type and target type are unrelated branches	Compiler error
Cast OK, actual object IS that type	Runs successfully
Cast syntactically OK, actual object is NOT that type	ClassCastException at runtime

Was the cast necessary? Yes (Y) if you needed the cast to access a method not in the declared type. No (N) if the method was already accessible without it.

Examples (given Loud ld = new WindInstrument("Sax", "Brass")):

```
ld.amplify(7) // OK – amplify is in Loud → runs WindInstrument.amplify → "Wamp7-"
ld.play() // COMPILER ERROR – play() not in Loud
((Instrument) ld).play() // OK, cast necessary (Y); runs WindInstrument.play() → "Blow Brass"
((Guitar) ld).amplify(1) // Compiles, but actual object is WindInstrument, not Guitar
                        // → ClassCastException at runtime
```

Tracing Chained Method Calls

When one method calls another (e.g., m2() calls m1()), the inner call also resolves dynamically — always use the actual object's most overridden version. super.method() jumps up exactly one level.

```
// actual object is Guitar; ins declared as Instrument
ins.play() → Guitar.play() prints "Chord!"
```

```
// actual object is WindInstrument; ld declared as Loud
((Instrument)ld).play() → WindInstrument.play() returns "Blow Brass" (printed by caller)
```

Part 4: Writing a Class Hierarchy

Key Concepts

Abstract class: Use abstract when the class should never be instantiated directly and/or has methods subclasses must implement.

Interface: A contract of method signatures. A class implements it and must provide all declared methods. A class can implement multiple interfaces but extend only one class.

super () in constructors: Always call super (args) as the first line to pass data up — avoids duplicating field assignments.

Template

```
abstract class Base {
    private Type field;
    public Base(Type f) { this.field = f; }
    public abstract ReturnType abstractMethod();
    public void concreteMethod() { /* can call abstractMethod() */ }
}

interface Doable {
    void doThing(int n);
}

class Child extends Base {
    private Type2 extra;
    public Child(Type f, Type2 e) {
        super(f); // ← always first
        this.extra = e;
    }
    public ReturnType abstractMethod() { return ...; }
}

class GrandChild extends Child implements Doable {
    private Type3 more;
    public GrandChild(Type f, Type2 e, Type3 m) {
        super(f, e); // ← chain up
        this.more = m;
    }
    public ReturnType abstractMethod() { return ...; } // override
    public void doThing(int n) { /* interface method */ }
}
```

Common Mistakes

- Forgetting abstract on the class when it has abstract methods.
- Not calling super () first in a subclass constructor.
- Putting instance variables in an interface (interfaces only declare methods).
- Re-declaring parent fields in a subclass — use super () instead.
- Forgetting to implement all interface methods in a concrete subclass.

Quick Example — Vehicle Hierarchy

```

public abstract class Vehicle {
    private String make;
    public Vehicle(String make) { this.make = make; }
    public abstract String fuelType();
    public void describe() {
        System.out.println("Vehicle: " + make + ", Fuel: " + fuelType());
    }
}

public interface Electric { void charge(int minutes); }

public class Car extends Vehicle {
    private int doors;
    public Car(String make, int doors) { super(make); this.doors = doors; }
    public String fuelType() { return "Gasoline"; }
}

public class ElectricCar extends Car implements Electric {
    private double batteryKWh;
    public ElectricCar(String make, int doors, double batteryKWh) {
        super(make, doors); this.batteryKWh = batteryKWh;
    }
    public String fuelType() { return "Electric"; }
    public void charge(int minutes) {
        System.out.println("Charging for " + minutes + " min");
    }
}

```

Note: `describe()` in `Vehicle` calls `fuelType()`. Due to dynamic dispatch, calling it on an `ElectricCar` prints "Electric" even though `describe()` is defined in `Vehicle`.

Part 5: Writing `equals()` and `compareTo()`

`equals()` Pattern

In a base class:

```

public boolean equals(Object obj) {
    if (this == obj) return true; // same object in memory
    if (!(obj instanceof MyClass)) return false; // wrong type
    MyClass other = (MyClass) obj; // safe cast
    return Math.abs(this.value - other.value) < TOL; // compare doubles
    // For non-doubles: return this.field.equals(other.field);
}

```

In a subclass (use `super.equals()` to check parent fields first):

```

public boolean equals(Object obj) {
    if (this == obj) return true;
    if (!(obj instanceof SubClass)) return false;
    if (!super.equals(obj)) return false; // check parent fields
    SubClass other = (SubClass) obj;
    return this.extraField == other.extraField; // check subclass fields
}

```

`compareTo()` Pattern

Returns **negative** if `this < other`, **0** if equal, **positive** if `this > other`.

For doubles (never subtract directly — use a tolerance check):

```

public int compareTo(MyClass other) {
    double diff = this.value - other.value;
    if (Math.abs(diff) < TOL) return 0;
    return diff < 0 ? -1 : 1;
}

```

```
}
```

Subclass with a tie-break:

```
// Rectangle: primary = area, tie-break = perimeter
public int compareTo(Rectangle other) {
    double areaDiff = this.getArea() - other.getArea();
    if (Math.abs(areaDiff) >= TOL) {
        return areaDiff < 0 ? -1 : 1;
    }
    // tie-break by perimeter
    double perimDiff = this.getPerimeter() - other.getPerimeter();
    if (Math.abs(perimDiff) < TOL) return 0;
    return perimDiff < 0 ? -1 : 1;
}
```

Comparable **setup**: When a class implements Comparable<T>, the signature is `public int compareTo(T other)` — the parameter is already the right type, no cast needed.

Quick Reference: Error Decision Tree

ASSIGNMENT: `LeftType var = new RightType()`

Is LeftType a supertype of RightType?

No → COMPILER ERROR (bad assignment)

Yes → OK. Is RightType abstract or an interface?

Yes → COMPILER ERROR (cannot instantiate)

No → runs fine

METHOD CALL: `variable.method()`

Is method() in the DECLARED type of variable?

No → COMPILER ERROR

Yes → Compiles. At runtime, JVM runs the most overridden version in the ACTUAL type.

CAST: `((TargetType) variable).method()`

Is the declared type related to TargetType (same hierarchy)?

No → COMPILER ERROR

Yes → Is the actual object really a TargetType?

Yes → Runs OK. Was the cast needed?

No → Unnecessary (extraneous) cast

Yes → Necessary cast

No → ClassCastException at runtime